

■ Contratos y Especificaciones Técnicas – Prueba Técnica

Este documento describe los contratos (endpoints y payloads) y las especificaciones técnicas del proyecto backend y frontend.

Autenticación

- POST /api/v1/auth/login – body `{ email: string, password: string }` → `{ token: string, refreshToken: string, user: {id,email,role} }`
- POST /api/v1/auth/refresh – body `{ refreshToken: string }` → `{ token: string }`

CRUD de recursos principales

Cada recurso expone GET/POST/PUT/DELETE con body JSON y responde `{ data, meta }`.

Customers

- GET /api/v1/customers – lista paginada `{ data: Customer[], meta: { count, page, pageSize } }`
- POST /api/v1/customers – crea `{ name, email }`
- PUT /api/v1/customers/:id – actualiza `{ name?, email? }`
- DELETE /api/v1/customers/:id – elimina

Products

- GET /api/v1/products – lista `{ data: Product[], meta }`
- POST /api/v1/products – crea `{ name, sku, price, category }`
- PUT /api/v1/products/:id – actualiza
- DELETE /api/v1/products/:id – elimina

Orders

- GET /api/v1/orders – lista `{ data: Order[], meta }` con include `customer` y `items`
- POST /api/v1/orders – crea `{ customer\_id, order\_date, status, payment\_method, total\_amount }`
- PUT /api/v1/orders/:id – actualiza
- DELETE /api/v1/orders/:id – elimina

OrderItems

- GET /api/v1/order-items – lista `{ data: OrderItem[], meta }` con include `product` y `order`
- POST /api/v1/order-items – crea `{ order\_id, product\_id, quantity, unit\_price }`
- PUT /api/v1/order-items/:id – actualiza
- DELETE /api/v1/order-items/:id – elimina

Especificaciones Técnicas – Backend

- Node.js + Express.
- Sequelize ORM con migraciones y seeders (convención snake_case en BD).
- MySQL 8.0 en Docker.
- Autenticación JWT con access token y refresh token.

- Middleware de autorización para proteger rutas.
- Middleware central de manejo de errores con respuesta JSON estándar.
- Seeds iniciales para usuario admin, clientes, productos y pedidos.

Especificaciones Técnicas – Frontend

- Angular 18 con Angular Material puro (sin Bootstrap/Tailwind).
- Componente Login con formulario reactivo y `MatSnackBar` para feedback.
- Dashboard Home con KPIs y gráficos `ngx-echarts` alimentados solo con endpoints CRUD.
- Interceptores HTTP para adjuntar `Authorization: Bearer` y manejar refresh token.
- Guard de autenticación para proteger rutas privadas.
- Dockerfile para construir y servir Angular con Nginx.

Infraestructura Docker

- Servicio MySQL con volumen persistente y usuario admin/jahinsurance*.
- Servicio backend Node/Express con conexión a MySQL.
- Servicio frontend Angular compilado y servido por Nginx en puerto 4200.
- Nginx configurado para proxy_pass /api al backend.